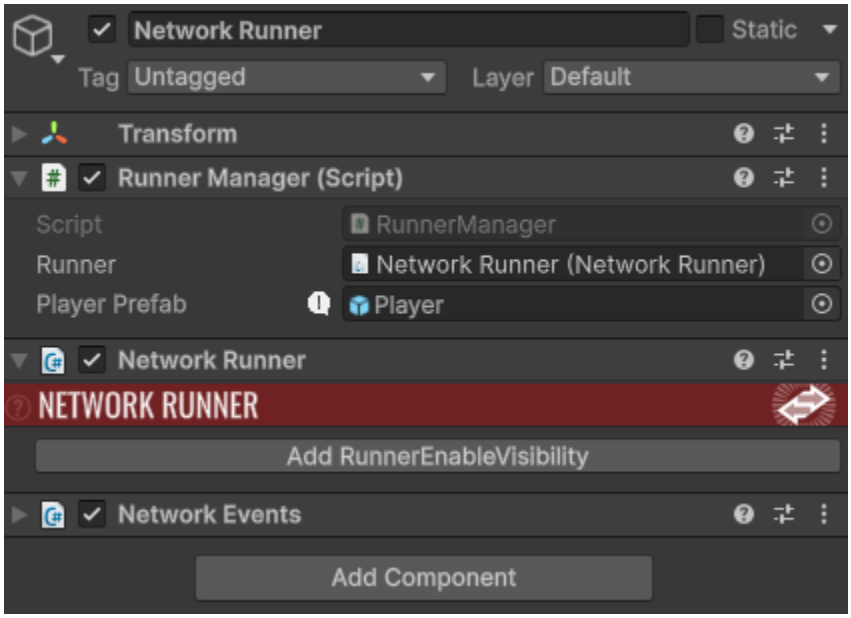
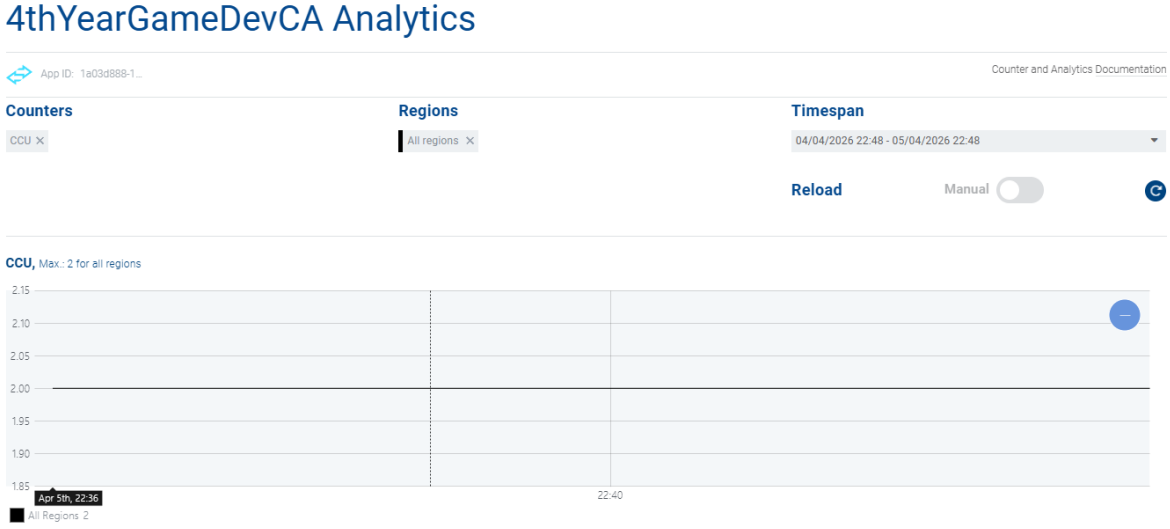
CA2 – Networking + Version Control

Repo: https://github.com/hillyleopard133/4th-year-game-dev/tree/main/Docs/3D%20Game%20dev

Session setup and spawn baseline




```
await runner.StartGame(new StartGameArgs()
{
    GameMode = GameMode.Shared,
    SessionName = "CA2Session",
    Scene = SceneRef.FromIndex(SceneManager.GetActiveScene().buildIndex),
    SceneManager = gameObject.AddComponent<NetworkSceneManagerDefault>()
}); // Task<StartGameResult>
```



Feature Overview

The implemented feature is a basic networked player spawning and ownership system, where each connected client controls their own player instance in a shared Fusion session.

Player objects are spawned when a client joins the session, and ownership is assigned using Fusion's authority model to ensure correct input handling and state consistency.

Authority Implementation

Authority is enforced using Fusion's built-in ownership checks:

```
public void OnPlayerJoined(NetworkRunner runner, PlayerRef player)
{
    if (runner.IsServer)
    {
        runner.Spawn(playerPrefab, new Vector3(0, 1, 0), Quaternion.identity, player);
    }
}
```

- **Input Authority**
  Each spawned player is assigned InputAuthority via the PlayerRef parameter. This ensures that only the owning client can provide input for their own player object.
- **State Authority**
  StateAuthority is assigned implicitly by Fusion in Shared Mode. The peer that spawns the object initially holds authority over its state, while updates are synchronised across all connected clients.

Synchronisation Approach

This implementation uses Fusion's state replication model using [Networked] properties (or intended use of them) rather than RPC-based updates.

The reason for choosing a [Networked] approach is that it provides continuous state synchronisation of gameplay-relevant values such as position and movement, which is more suitable for real-time player control than discrete RPC calls.

RPCs are better suited for event-based actions (e.g., shooting, interactions), whereas [Networked] variables ensure smooth ongoing replication of player state across clients.

Summary

The feature implements a foundational authority-based multiplayer system using Photon Fusion's Shared Mode. InputAuthority is correctly assigned per player, ensuring isolated input handling, while StateAuthority governs object state replication. The system is designed

to avoid desynchronisation by enforcing strict authority checks and using networked state replication for continuous updates.

Unreal Replication Terminology (Fusion vs Unreal Engine 5)

Photon Fusion and Unreal Engine 5 both implement authoritative multiplayer networking, but they differ significantly in how authority and replication are structured and exposed to the developer.

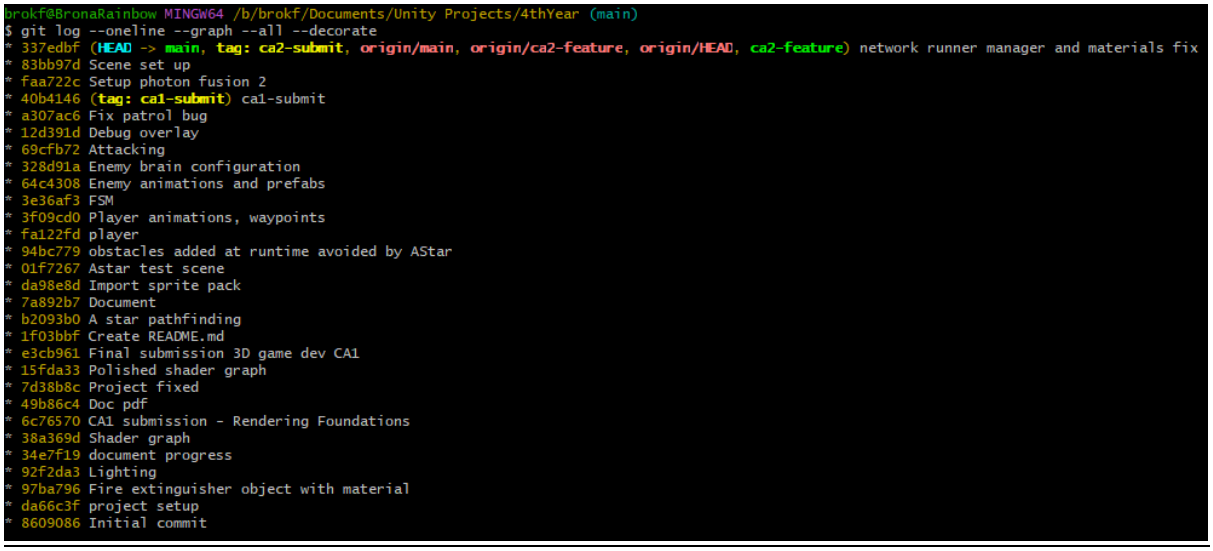
In Fusion, StateAuthority determines which peer is responsible for simulating and writing the state of a network object, while InputAuthority determines which client is allowed to provide input for that object. This maps conceptually to Unreal Engine 5's replication model where the Server holds authoritative control over game state, the Owning Client is responsible for local input, and Simulated Proxies are remote clients that receive replicated state updates but do not directly control the object. In this mapping, Fusion's StateAuthority is most similar to the Unreal Server role, while InputAuthority corresponds to the Owning Client that sends input to the server for validation and processing.

Fusion's [Networked] properties are used to automatically synchronise state variables across all clients in a session. The Unreal equivalent is a Replicated UPROPERTY, which is marked with the Replicated specifier in C++. Both systems ensure that changes to variables such as position, health, or velocity are propagated across the network. However, Unreal typically requires explicit replication setup and may involve additional functions such as GetLifetimeReplicatedProps, whereas Fusion handles replication more directly through its network simulation model.

For event-based communication, Fusion uses RPCs (Remote Procedure Calls), which allow functions to be executed across the network. This maps directly to Unreal's RPC system, which includes Server RPCs, Client RPCs, and NetMulticast RPCs. Server RPCs in Unreal are used when a client requests the server to perform an action (e.g., firing a weapon or validating input). Client RPCs are used by the server to send targeted information back to a specific client. NetMulticast RPCs are used when the server needs to broadcast an event to all clients simultaneously, such as an explosion or global animation trigger. In Fusion, RPC usage is more unified, but the conceptual purpose remains similar: distinguishing between state updates and discrete events.

A key conceptual difference between Fusion and Unreal Engine networking is the level of abstraction in authority handling. Fusion exposes authority explicitly through StateAuthority and InputAuthority at the object level, allowing more flexible peer-based simulation models such as Shared Mode. In contrast, Unreal Engine enforces a stricter server-authoritative architecture where the server is always the ultimate source of truth. This means Fusion can support more distributed simulation patterns, while Unreal's model is more rigid but often easier to reason about in traditional client-server setups.

Version control evidence



I couldn't get the graph to load for me so I went with a screenshot from git bash instead, showing the use of the ca2-feature branch and ca2-submit tag.

I used a dedicated feature branch (ca2-feature) for the CA2 networked system, where development of Fusion networking functionality was carried out independently from the main branch. Once the feature was stable, it was merged back into main and tagged with ca2-submit to mark the final submission state. This approach helped isolate experimental changes and prevented unstable networking code from affecting the main project. A key lesson learned was the importance of committing incrementally while working on a feature branch, as it makes debugging easier and provides a clear development history for assessment and rollback if needed.

Learning Journal and evidence log

Weekly evidence log

Week	Dates	What I worked on	Key commits	Evidence ref	Blocker / resolution
1–4	Feb 15–17	CA1 rendering pipeline: project setup, lighting, shader graph development, material assignment, and final CA1 submission	Initial commit, project setup, lighting, shader graph, CA1 submission	CA1 doc	Shader graph inconsistencies resolved through iterative node adjustments
5–7	Feb 18–Apr 4	No version-controlled development committed.	—	—	—
8	Apr 5	Introduced Photon Fusion 2, initial project setup, scene preparation	setup photon fusion 2, scene setup	above	App setup issues resolved via correct project configuration
9	Apr 6	Implemented NetworkRunner manager and initial networking/material fix; beginning of multiplayer session structure	network runner manager and materials fix	above	Session instability resolved by correcting runner configuration

Critical discussion of technical choices

The feature scope for CA2 was deliberately limited to a minimal multiplayer foundation consisting of session creation, player joining, and correct networked player spawning using Photon Fusion. This scope was appropriate for a two-week sprint (Weeks 8–9) because it focused on validating core networking systems such as the NetworkRunner lifecycle, session connectivity, and authority assignment. More complex gameplay features such as combat, physics interactions, or inventory systems were intentionally not attempted, as they would significantly increase debugging complexity and risk destabilising the core requirement of achieving a stable two-client session. Prioritising a simple but reliable baseline ensured that the system met the assessment requirement of consistent spawning and connectivity before any feature expansion.

For synchronisation, the implementation primarily relies on Fusion's [Networked] model and built-in object replication rather than using RPCs for continuous state updates. [Networked] properties are best suited for persistent state that must remain consistent across clients, such as object transforms or gameplay-relevant variables. RPCs are more appropriate for discrete, event-based actions such as triggering interactions or one-off events. In this project, player spawning and session presence are continuous states rather than events, so relying on Fusion's networked object replication model was more appropriate than using RPC calls. RPCs were considered but not required for the baseline system, as they would be better suited for future extensions such as interaction triggers or gameplay actions.

A key authority decision occurred in the OnPlayerJoined implementation. Player objects were spawned using the incoming PlayerRef, which determines InputAuthority. Initially, incorrect callback ordering and runner setup caused inconsistent behaviour where the second client either failed to spawn a player or did not correctly receive control of its object. This indicated that authority assignment and callback registration were not aligned correctly in the execution order. The issue was resolved by ensuring runner.AddCallbacks(this) was called before StartGame() and verifying that runner.Spawn() correctly passed the joining PlayerRef. This ensured each client received exclusive InputAuthority over its own player object while maintaining a consistent shared simulation state across all clients.

If the feature were to be redesigned, the main change would be separating networking responsibilities into distinct systems rather than using a single RunnerManager class. In the current implementation, session setup, scene management, and spawning logic are tightly coupled, which made early debugging more difficult when diagnosing session or authority issues. This is reflected in the Apr 6 commit where multiple fixes were required at once ("network runner manager and materials fix"). Splitting responsibilities into dedicated components such as a bootstrap manager and a player spawner would improve clarity, reduce coupling, and make authority-related issues easier to isolate during development.

Lessons learned

- During initial Fusion setup, I assumed the NetworkRunner would automatically handle session stability once the App ID was set. However, early testing showed inconsistent session joining and missing debug output. I learned that proper callback registration order (especially runner.AddCallbacks(this) before StartGame) is critical, otherwise events like OnPlayerJoined will not reliably fire.
- When implementing player spawning, I initially placed authority logic without fully validating how PlayerRef maps to InputAuthority in runner.Spawn(). This resulted in situations where player objects appeared inconsistently between clients or were not controlled correctly. Ensuring that the joining PlayerRef is passed directly into Spawn() resolved the issue and made each client correctly own its player instance.
- I initially underestimated how sensitive Fusion is to configuration consistency between scene setup and runtime runner configuration. A misconfigured NetworkRunner setup led to repeated disconnects ("DisconnectByClientLogic") during early testing. Fixing this required verifying scene loading, session setup, and ensuring the NetworkSceneManager was correctly assigned.
- My debugging process showed that most multiplayer issues were not caused by logic errors in scripts, but by setup order and configuration mismatches. This shifted my approach from "code-first debugging" to checking Fusion setup requirements (runner configuration, callbacks, scene management) before changing gameplay logic.
- The commit history also showed that I tended to bundle multiple fixes into single commits during early CA2 work. After encountering debugging difficulty in the Apr 6 "network runner manager and materials fix" commit, I realised smaller, incremental commits would have made it easier to isolate authority and spawning issues earlier.